AWS Full Stack Deployment Mastering Guide -- Complete Professional Handbook

---

Author: Senior Cloud Architect & DevOps Engineer **Target Audience:** Backend/Full Stack developers with 2-4 years of experience **Prerequisites:** Spring Boot, React, basic Linux **Goal:** Master deploying real applications on AWS and clear cloud engineering interviews

---

Table of Contents

1. Introduction to Cloud Computing
2. AWS Overview
3. AWS Account Setup (Practical)
4. EC2 (Virtual Servers)
5. Elastic Load Balancing (ELB)
6. Auto Scaling
7. S3 (Simple Storage Service)
8. RDS (Relational Database Service)
9. Amazon VPC (Networking)
10. AWS IAM (Identity and Access Management)
11. AWS Lambda (Serverless Computing)
12. Docker Deployment (ECS & ECR)
13. CI/CD Pipeline (CodePipeline)
14. AWS CloudWatch (Monitoring)
15. Full Production Architecture
16. Interview Preparation (50+ AWS Questions)
17. AWS Production Best Practices

Part 1: Introduction to Cloud Computing

1.1 What is Cloud Computing?

Cloud computing is the on-demand delivery of IT resources (compute power, database storage, applications, and other IT resources) over the Internet with pay-as-you-go pricing. Instead of buying, owning, and maintaining physical data centers and servers, companies can access computing power and storage on an as-needed basis from a cloud provider like Amazon Web Services (AWS).

1.2 Why Companies Use Cloud Computing

Before the cloud, companies had to forecast their IT needs months or years in advance. They had to buy hardware, rack and stack servers, and physically wire data centers.

Key benefits of the cloud:

- **Agility:** Cloud gives you easy access to a broad range of technologies so you can innovate faster.
- **Elasticity:** You don't have to over-provision resources up front to handle peak levels of business activity. You can provision the amount of resources you actually need and scale them up or down instantly.
- **Cost Savings:** The cloud allows you to trade fixed expenses (such as data centers and physical servers) for variable expenses, and only pay for IT as you consume it.
- **Deploy Globally in Minutes:** You can easily deploy your application in multiple physical locations around the world with just a few clicks.

1.3 Traditional Hosting vs. Cloud Infrastructure

Feature	Traditional On-Premises	Cloud Infrastructure
Capital Expense	High initial investment (CapEx)	Zero upfront cost, pay-as-you-go (OpEx)
Scaling	Manual, slow (weeks/months)	Automated, instant (Auto Scaling)
Maintenance	Your team manages hardware/power	Cloud provider manages physical hardware
Security	You build physical security fencing	Provider has military-grade data centers

Deployment	Slow, manual rack stacking	Infrastructure as Code (Terraform/CloudFormation)

1.4 Cloud Service Models Explained

Cloud computing comes in three primary service models, representing different levels of abstraction and shared responsibility.

1. IaaS (Infrastructure as a Service)

What it is: You rent raw IT infrastructure--servers, virtual machines, storage, networks, and operating systems. You are responsible for managing the OS, middleware, and applications. **Example:** AWS EC2 (Elastic Compute Cloud), S3, VPC. **Real-World Use Case:** A company wants total control over their Linux environment to run a custom-compiled database engine.

2. PaaS (Platform as a Service)

What it is: The provider manages the underlying infrastructure (usually hardware and operating systems), allowing you to focus purely on the deployment and management of your applications. **Example:** AWS Elastic Beanstalk, Heroku, AWS RDS (Relational Database Service). **Real-World Use Case:** A developer wants to deploy a Spring Boot `.jar` file without having to install Java, configure Tomcat, or patch the underlying Linux OS.

3. SaaS (Software as a Service)

What it is: A complete product that is run and managed by the service provider. You only use the software, usually via a web browser. **Example:** Gmail, Salesforce, DropBox. **Real-World Use Case:** A company uses Gmail for corporate email instead of managing their own Microsoft Exchange servers.

```
+-------------------------------------------------------------+
|               Cloud Service Models Comparison               |
+-------------------------------------------------------------+
|                                                             |
|   You Manage             You Manage           You Manage    |
|   [ ] Applications       [ ] Applications     [x] Nothing!  |
|   [ ] Data               [ ] Data             [x] Nothing!  |
|   [ ] Runtime            [x] AWS Manages      [x] AWS       |
|   [ ] Middleware         [x] AWS Manages      [x] AWS       |
|   [ ] O/S                [x] AWS Manages      [x] AWS       |
|   [x] AWS Manages        [x] AWS Manages      [x] AWS       |
|   [x] AWS Manages        [x] AWS Manages      [x] AWS       |
|                                                             |
|       IaaS                   PaaS                 SaaS      |
```

```
|     (e.g., EC2)        (e.g., Beanstalk)     (e.g., Gmail)  |
+-------------------------------------------------------------+
```

Part 2: AWS Overview

2.1 What is AWS?

Amazon Web Services (AWS) is the world's most comprehensive and broadly adopted cloud platform, offering over 200 fully featured services from data centers globally. It provides highly reliable, scalable, low-cost infrastructure platforms in the cloud that power hundreds of thousands of businesses in 190 countries.

2.2 Global Infrastructure

AWS operates a massive global network infrastructure. Understanding this topology is critical for designing highly available and fault-tolerant full-stack applications.

Regions

What they are: A Region is a physical, geographical location in the world where AWS clusters its data centers (e.g., `us-east-1` in N. Virginia, `ap-south-1` in Mumbai). **Why they matter:** You choose a Region based on data compliance laws, latency (closeness to your users), and service availability. **Rule of Thumb:** Regions are completely isolated from one another. If an earthquake destroys a Region, applications in other Regions remain completely unaffected.

Availability Zones (AZs)

What they are: Every Region consists of multiple, isolated, and physically separate AZs (usually 3 or more). An AZ is one or more discrete data centers with redundant power, networking, and connectivity. **Why they matter:** To build a highly available Spring Boot backend, you must deploy your application instances across *multiple* AZs within a Region. If one AZ goes down (due to power failure, flood), the other AZs keep serving traffic.

Edge Locations

What they are: Datacenters used specifically to cache content closer to end users to reduce latency. There are vastly more Edge Locations than Regions worldwide. **Why they matter:** Edge Locations are the foundation of AWS CloudFront (Content Delivery Network). When you deploy a compiled React application (HTML/CSS/JS), you distribute it to Edge Locations so users globally can download the UI in milliseconds.

2.3 Global Architecture Diagram

```
+-----------------------------------------------------------------+
|                        AWS GLOBAL NETWORK                       |
+-----------------------------------------------------------------+
|                                                                 |
|   REGION: us-east-1 (N. Virginia)                               |
|   +-------------------------------------------------------+     |
|   |  AZ: us-east-1a         AZ: us-east-1b                |     |
|   |  +----------------+     +----------------+            |     |
|   |  | Data Center 1  |     | Data Center 3  |            |     |
|   |  | Data Center 2  |     | Data Center 4  |            |     |
|   |  +----------------+     +----------------+            |     |
|   |          \                      /                      |     |
|   |           \                    /                       |     |
|   |            \                  /                        |     |
|   |        High-speed, low-latency fiber                   |     |
|   +-------------------------------------------------------+     |
|                                                                 |
|   EDGE LOCATIONS (Global Caching via CloudFront)                |
|   [London]  [Tokyo]  [Mumbai]  [Sydney]  [Sao Paulo]            |
|                                                                 |
+-----------------------------------------------------------------+
```

Part 3: AWS Account Setup (Practical)

Before deploying a Spring Boot and React stack, the very first step in AWS is establishing a secure administrative account.

3.1 Create an AWS Account

1. Go to `aws.amazon.com` and create an AWS account.
2. Provide billing information (AWS requires a credit card even for the Free Tier).
3. The email you use to register becomes your **Root Account**.

3.2 Securing the Root Account

Common Mistake: *Never* use your Root Account for daily development or application deployment. The Root Account has unrestricted, unstoppable access to everything, including billing.

1. Log into the AWS Management Console as the Root user.
2. Search for **IAM** (Identity and Access Management).
3. Under the IAM Dashboard, locate the "Add MFA" warning.
4. **Enable MFA (Multi-Factor Authentication):** Register a virtual MFA device (like Google Authenticator or Authy on your phone). Scan the QR code, enter two consecutive tokens.
5. Once MFA is enabled, log out, and you will almost never use the Root account again.

3.3 Create an IAM Admin User for Daily Use

Instead of the Root account, you interact with AWS using an IAM User.

1. In the IAM Dashboard (logged in as Root, one last time), click **Users** -> **Add users**.
2. **User name:** `admin-user` (or your name).
3. **Select AWS access type:** Check both "Programmatic access" (for CLI/API) and "AWS Management Console access".
4. **Set permissions:** Click "Attach existing policies directly" -> Select `AdministratorAccess`.
5. **Review and create.**
6. **CRITICAL:** Download the `.csv` file containing the `Access key ID` and `Secret access key`. You will need this for the AWS CLI. *Never share this or commit it to GitHub.*

3.4 Configure the AWS CLI

To deploy Docker containers, configure CI/CD loops, or script infrastructure, developers use the AWS CLI.

Step 1: Install the AWS CLI

For Windows (PowerShell):

```
msiexec.exe /i https://awscli.amazonaws.com/AWSCLIV2.msi
```

For macOS (Terminal):

```
curl "https://awscli.amazonaws.com/AWSCLIV2.pkg" -o "AWSCLIV2.pkg"
sudo installer -pkg AWSCLIV2.pkg -target /
```

For Linux:

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o
"awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
```

Step 2: Configure Credentials

Open your terminal and run:

```
aws configure
```

You will be prompted to enter the credentials from the CSV file you downloaded earlier:

```
AWS Access Key ID [None]: AKIAIOSFODNN7EXAMPLE
AWS Secret Access Key [None]: wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
Default region name [None]: us-east-1
Default output format [None]: json
```

Step 3: Verify Connection

Test that your CLI can talk to AWS by listing your IAM users:

```
aws iam list-users
```

If it returns a JSON block with your `admin-user`, your setup is complete and secure. You are now ready to start deploying real applications!

Part 4: EC2 (Virtual Servers)

4.1 What is AWS EC2?

Amazon Elastic Compute Cloud (EC2) provides scalable computing capacity in the AWS cloud. Simply put, it allows you to rent virtual servers (called instances) on which you can install your operating system, software, and deploy your Spring Boot or React applications.

4.2 Why Does EC2 Exist?

Before EC2, launching a new server meant ordering physical hardware, waiting weeks for delivery, racking the server, installing the OS, and plugging in network cables. EC2 eliminates this overhead. You can provision hundreds of virtual servers with varying capacities in seconds and shut them down when no longer needed, paying only for the time they run.

4.3 Key EC2 Concepts

- **Instance Types:** EC2 instances come in different families optimized for specific use cases (e.g., Compute Optimized, Memory Optimized, General Purpose). Example: `t3.micro` is good for small web apps, `c5.large` is for CPU-heavy processing.
- **AMI (Amazon Machine Image):** A template containing the software configuration (OS, application server, applications) required to launch an instance. Example: Amazon Linux 2, Ubuntu 22.04.
- **Security Groups:** A virtual firewall that controls inbound and outbound traffic to your EC2 instance. You explicitly open ports (like port 22 for SSH, port 8080 for Spring Boot).
- **Key Pairs:** Used to prove your identity when connecting to an EC2 instance. AWS stores the public key, and you store the private key (`.pem` file) locally to SSH into the box.
- **Elastic IP:** A static IPv4 address designed for dynamic cloud computing. If an EC2 instance restarts, its public IP changes. An Elastic IP remains constant.

4.4 Real-World Production Use Cases

- Hosting a legacy monolithic Spring Boot application.
- Standing up a dedicated Jenkins or GitLab CI/CD server.
- Running background batch processing jobs using Spring Batch.
- Deploying a self-managed database (if not using RDS).

4.5 Practical Implementation: Deploying Spring Boot on EC2

This guide deploys a compiled Spring Boot `.jar` file exposed on port 8080.

Step 1: Launch the EC2 Instance

1. Open the AWS Console and search for **EC2**.
2. Click **Launch Instance**.
3. **Name:** Give your instance a name like `SpringBoot-Backend`.
4. **AMI:** Select **Amazon Linux 2023 AMI**.
5. **Instance Type:** Select `t2.micro` or `t3.micro` (Free Tier eligible).
6. **Key Pair:** Click **Create new key pair**. Name it `spring-boot-key`, choose `RSA` and `.pem`. Download and save it securely.

Step 2: Configure the Security Group

We must allow SSH (to access the server) and HTTP/8080 (for our Spring Boot API).

1. Under **Network settings**, click **Edit**.
2. **Auto-assign Public IP:** Ensure "Enable".
3. **Inbound Security Group Rules:**
   - **Type:** SSH, **Protocol:** TCP, **Port Range:** 22, **Source:** Anywhere/My IP
   - **Type:** Custom TCP, **Protocol:** TCP, **Port Range:** 8080, **Source:** Anywhere
4. Click **Launch Instance**. Wait ~30 seconds for it to start.

Step 3: Connect via SSH

Find the public IP address of your new instance from the EC2 Dashboard.

Open your terminal (on Mac/Linux) or PowerShell (Windows). Change the permissions of your `.pem` key (Mac/Linux only):

```
chmod 400 spring-boot-key.pem
```

Now, SSH into the instance:

```
ssh -i "spring-boot-key.pem" ec2-user@<YOUR-EC2-PUBLIC-IP>
```

Step 4: Install Java 17

Once logged into your EC2 terminal:

```
sudo yum update -y
sudo yum install java-17-amazon-corretto -y
java -version
```

Step 5: Upload and Run the Spring Boot JAR

Open a **separate** local terminal (do not run this inside the EC2 SSH session) to copy your built Spring Boot `.jar` to the server using `scp`:

```
scp -i "spring-boot-key.pem" target/my-spring-app-0.0.1-SNAPSHOT.jar ec2-user@<YOUR-EC2-PUBLIC-IP>:~/app.jar
```

Switch back to your EC2 SSH terminal and run the app:

```
# Run the application in the background using nohup
nohup java -jar app.jar > app.log 2>&1 &
```

Your Spring Boot application is now running globally! Test it by opening a browser and navigating to `http://<YOUR-EC2-PUBLIC-IP>:8080`.

4.6 Common Mistakes Developers Make

- **Losing the Key Pair:** If you lose the `.pem` file, you cannot SSH into your instance. AWS does not store a copy.
- **Wide Open Security Groups:** Leaving port 22 (SSH) open to `0.0.0.0/0` (the entire internet) is a massive security risk. Restrict it to your personal IP.
- **Hardcoding AWS Credentials:** Never hardcode AWS keys inside your Spring Boot `application.properties`. Use IAM Roles instead.

4.7 Interview Questions

Q: What is the difference between an On-Demand instance and a Spot instance? *Answer:* On-Demand instances have fixed, predictable pricing, ideal for uninterrupted workloads. Spot instances use spare AWS capacity and provide massive discounts (up to 90%), but AWS can terminate them with a 2-minute warning. They are best for fault-tolerant background jobs.

Q: If you stop and start an EC2 instance, what happens to its IP address? *Answer:* Its public IP address will change unless an Elastic IP is attached. The private IP address remains the same.

Part 5: Elastic Load Balancing (ELB)

5.1 What is an Elastic Load Balancer?

An Elastic Load Balancer (ELB) automatically distributes incoming application traffic across multiple targets, such as EC2 instances, containers, or IP addresses, in one or more Availability Zones.

5.2 Why Does It Exist?

If you deploy your Spring Boot backend to a single EC2 instance, you have two massive problems:

1. **Single Point of Failure:** If the instance crashes, your entire API goes offline.
2. **Scalability Ceiling:** A single instance can only handle so much CPU load before slowing down.

By placing a Load Balancer in front of *multiple* EC2 instances, traffic gets distributed evenly. If one instance crashes, the LB detects the failure (via health checks) and reroutes traffic to healthy instances.

5.3 Types of Load Balancers

- **Application Load Balancer (ALB):** Operates at Layer 7 (HTTP/HTTPS). Best for web applications like Spring Boot APIs and React apps. It can route traffic based on URL paths (e.g., `/api/v1/*` goes to backend group, `/static/*` goes to frontend group).
- **Network Load Balancer (NLB):** Operates at Layer 4 (TCP/UDP). Best for ultra-high performance, gaming servers, or handling millions of requests per second with microsecond latency.

5.4 Architecture Diagram

```
+-------------------------------------------------------------+
|               APPLICATION LOAD BALANCER (ALB)               |
+-------------------------------------------------------------+
|                                                             |
|                         [ Users ]                           |
|                             |                               |
|                     (HTTP Requests)                         |
|                             v                               |
|                +-------------------------+                  |
|                | Application Load Balancer|                  |
|                +-------------------------+                  |
|                    /                   \                    |
```

```
|                  /                    \                    |
|          +--------------+        +--------------+          |
|          | AZ: us-east-1a|       | AZ: us-east-1b|           |
|          | EC2 Instance 1|       | EC2 Instance 2|           |
|          | (Active)      |       | (Active)      |           |
|          +--------------+        +--------------+          |
|                                                             |
+-------------------------------------------------------------+
```

5.5 Practical Implementation: Setting up an ALB

Before starting, ensure you have *two* EC2 instances running your Spring Boot application in two different Availability Zones.

Step 1: Create a Target Group

A Target Group tells the ALB *where* to route traffic.

1. Go to EC2 Dashboard -> **Target Groups** -> **Create target group**.
2. **Target type:** Instances.
3. **Target group name:** `spring-boot-tg`.
4. **Protocol/Port:** HTTP / 8080 (Matches your Spring Boot app).
5. **Health check path:** `/actuator/health` (Assuming Spring Boot Actuator is enabled).
6. Register the two running EC2 instances into this group and click **Include as pending below**.

Step 2: Create the Load Balancer

1. Go to EC2 Dashboard -> **Load Balancers** -> **Create Load Balancer**.
2. Select **Application Load Balancer**.
3. **Name:** `spring-boot-alb`.
4. **Scheme:** Internet-facing.
5. **Network mapping:** Select at least two subnets in different Availability Zones.
6. **Security Groups:** Create/Select a security group that allows inbound HTTP on port 80.
7. **Listeners and routing:** For the HTTP:80 listener, select your `spring-boot-tg` target group.
8. Click **Create**.

Step 3: Test Scaling

Once the ALB is provisioned (takes ~2 minutes), copy its DNS Name (e.g., `spring-boot-alb-1234.us-east-1.elb.amazonaws.com`). Paste this into your browser. The ALB will seamlessly route traffic to your backend servers!

5.6 Interview Questions

Q: What is a Health Check in Load Balancing? *Answer:* A mechanism where the ALB continuously pings a specific endpoint (like `/health`) on the EC2 instances. If an instance stops responding with a HTTP 200 OK status, the ALB marks it as unhealthy and stops routing traffic to it.

Q: Does an ALB have a static IP address? *Answer:* No. An Application Load Balancer uses dynamic IPs that update over time as it scales internally to handle traffic volume. You must always refer to an ALB by its DNS Name. (Conversely, an NLB *can* use static Elastic IPs).

Part 6: Auto Scaling

6.1 What is Auto Scaling?

AWS Auto Scaling monitors your applications and automatically adjusts capacity to maintain steady, predictable performance at the lowest possible cost. When traffic spikes, Auto Scaling adds EC2 instances (Scale Out). When traffic drops, it removes instances (Scale In).

6.2 Why Does It Exist?

Traffic is rarely constant. An e-commerce site might have 1,000 users at 3 AM and 100,000 users at 5 PM. If you manually provision 50 servers to handle the 5 PM spike, you are wasting massive amounts of money at 3 AM. Auto Scaling ensures you only pay for exactly what you need at any given second.

6.3 Auto Scaling Concepts

- **Launch Template:** A blueprint that defines the configuration for the instances that Auto Scaling launches (AMI, Instance Type, Key Pair, Security Groups, and User Data script).
- **Auto Scaling Group (ASG):** The logical collection of EC2 instances managed together. It has a Minimum, Maximum, and Desired capacity.
- **Scaling Policies:** Rules that trigger scaling actions. (e.g., "Add 2 instances if average CPU > 70% for 5 minutes").

6.4 Practical Implementation: Auto Scaling a Spring Boot API

Step 1: Create a Launch Template

1. In the EC2 console, click **Launch Templates** -> **Create launch template**.
2. **Name:** `spring-boot-lt`.
3. Provide the AMI (e.g., Amazon Linux 2023), Instance type (`t3.micro`), Key pair, and Security Group (allow SSH and 8080).
4. **Advanced details -> User Data:** This is a crucial bash script that runs automatically when the EC2 instance boots. Add a script to install Java and download/run your jar on startup:

```
#!/bin/bash
yum update -y
yum install java-17-amazon-corretto -y
# Download jar from S3 (Requires IAM Role with S3 read access)
```

```
aws s3 cp s3://my-artifacts-bucket/app.jar /home/ec2-user/app.jar
nohup java -jar /home/ec2-user/app.jar > /home/ec2-user/app.log 2>&1 &
```

5. Click **Create launch template**.

Step 2: Create Auto Scaling Group

1. Click **Auto Scaling Groups** -> **Create Auto Scaling group**.
2. **Name:** `spring-boot-asg`.
3. Select your `spring-boot-lt` launch template.
4. Select your VPC and check all Availability Zones.
5. **Load Balancing:** Attach to the existing Application Load Balancer target group (`spring-boot-tg`) created in Part 5.
6. **Group Size:** Desired: 2, Minimum: 2, Maximum: 5.
7. **Scaling Policies:** Choose "Target tracking scaling policy". Metric: Average CPU utilization. Target value: 60%.

Step 3: Test Scaling

To test, you can SSH into an EC2 instance and run a stress tool to max out the CPU. Once the CPU artificially hits 100%, AWS Auto Scaling will detect the breach of the 60% policy, automatically provision a new EC2 instance, install Java via the User Data script, and attach it to the Load Balancer!

6.5 Interview Questions

Q: What is the difference between Vertical Scaling and Horizontal Scaling? Which does AWS Auto Scaling do? *Answer:* Vertical scaling (Scaling Up) means upgrading an existing server to a larger instance type (e.g., increasing RAM/CPU). Horizontal scaling (Scaling Out) means adding more servers of the same size to distribute load. AWS Auto Scaling groups perform **Horizontal Scaling**. Horizontal scaling combined with stateless REST APIs is the key to massive cloud architecture.

Q: If an instance launched by an ASG crashes, what happens? *Answer:* The ASG constantly performs health checks. If an instance goes down, the Desired Capacity drops below the target. The ASG automatically terminates the dead instance and spins up a brand new, identical instance to replace it, ensuring high availability.

Part 7: S3 (Simple Storage Service)

7.1 What is AWS S3?

Amazon Simple Storage Service (S3) is an object storage service offering industry-leading scalability, data availability, security, and performance. Unlike a file system (which stores data in folders) or block storage (which stores chunks of data on hard drives), S3 stores data as *objects* within globally unique containers called *buckets*.

7.2 Why Does S3 Exist?

Traditional hard drives fill up. If you build a social media application and users upload 5 terabytes of images, an EC2 EBS volume will eventually run out of space, forcing you to manually attach a new drive. With S3, storage is infinitely scalable. You never have to provision disk space. You pay only for what you use, and the data is automatically replicated across multiple servers for 99.999999999% durability.

7.3 S3 Core Concepts

- **Bucket:** The root-level container for objects stored in S3. Bucket names must be globally unique across all AWS accounts (e.g., `my-company-profile-images`).
- **Object:** The fundamental entity stored in Amazon S3. An object consists of data, metadata, and a key.
- **Key:** The unique identifier for an object within a bucket. Similar to a file path (e.g., `users/123/profile.jpg`).
- **Access Control:** By default, all S3 buckets are private. You use Identity and Access Management (IAM) policies, bucket policies, and Access Control Lists (ACLs) to manage permissions.

7.4 Real-World Production Use Cases

- **Asset Storage:** Storing user-uploaded profile pictures, PDF invoices, and video files.
- **Static Website Hosting:** Hosting the compiled HTML, CSS, and JS files of a React application.
- **Data Lakes:** Storing massive amounts of unstructured data as the foundation for big data analytics.
- **Backups:** Automatically exporting RDS database backups and EC2 snapshots.

7.5 Practical Implementation: Uploading Files from React to S3 via Spring Boot

Common Mistake: Never embed AWS credentials directly into the React frontend to upload to S3. Anyone can open Developer Tools and steal your AWS keys.

Best Practice: The React frontend asks the Spring Boot backend for a **Pre-signed URL**. The frontend then uses this URL to safely upload the file directly to S3 without passing the file through the backend server (which saves backend bandwidth and memory).

Step 1: Create the S3 Bucket

1. Open the AWS Console -> search for **S3** -> **Create bucket**.
2. **Bucket name:** `user-profile-images-2026` (Must be globally unique).
3. **Region:** `us-east-1` (Match your EC2 region).
4. **Block Public Access settings:** Clear the checkbox to unblock public access (only if we want these images to be publicly viewable on a website).
5. Click **Create bucket**.

Step 2: Configure Bucket Policy

We must attach a JSON policy that explicitly allows public read access.

1. Click on the bucket -> **Permissions** tab -> **Bucket Policy** -> Edit.
2. Paste the following policy (replace `YOUR-BUCKET-NAME`):

```json
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "PublicReadGetObject",
            "Effect": "Allow",
            "Principal": "*",
            "Action": "s3:GetObject",
            "Resource": "arn:aws:s3:::user-profile-images-2026/*"
        }
    ]
}
```

Step 3: Configure Spring Boot to Generate a Pre-signed URL

Add the AWS SDK library to your `pom.xml`:

```
<dependency>
    <groupId>software.amazon.awssdk</groupId>
    <artifactId>s3</artifactId>
    <version>2.20.0</version>
</dependency>
```

Create an S3 Service in Spring Boot:

```
@Service
public class S3PresignedService {

    private final S3Presigner presigner;

    public S3PresignedService() {
        this.presigner = S3Presigner.builder()
            .region(Region.US_EAST_1)
            // Credentials come from IAM Role attached to EC2 automatically
            .build();
    }

    public String generatePresignedUrl(String objectKey) {
        PutObjectRequest objectRequest = PutObjectRequest.builder()
            .bucket("user-profile-images-2026")
            .key(objectKey)
            .build();

        PutObjectPresignRequest presignRequest =
PutObjectPresignRequest.builder()
            .signatureDuration(Duration.ofMinutes(10)) // URL expires in 10
mins
            .putObjectRequest(objectRequest)
            .build();

        PresignedPutObjectRequest presignedRequest =
presigner.presignPutObject(presignRequest);
        return presignedRequest.url().toString();
    }
}
```

Step 4: Uploading directly from React

In your React application:

```
const uploadFile = async (file) => {
  // 1. Get the pre-signed URL securely from our backend
  const response = await fetch('https://api.myapp.com/api/s3/generate-url?
fileName=' + file.name);
  const { uploadUrl } = await response.json();

  // 2. Upload the file directly to S3 using the pre-signed URL
  await fetch(uploadUrl, {
    method: 'PUT',
    body: file,
    headers: {
      'Content-Type': file.type,
    },
  });

  console.log('File successfully uploaded securely to S3!');
};
```

7.6 Interview Questions

Q: What is the difference between S3 Standard and S3 Glacier? *Answer:* S3 Standard offers high durability and immediate access for frequently accessed data (like website images). S3 Glacier is a much cheaper archive storage tier used for data that is rarely accessed (like compliance logs). Retrieving data from Glacier can take minutes to hours.

Q: S3 provides 99.999999999% (11 9's) durability. What does durability actually mean? *Answer:* Durability refers to data preservation. If you store 10,000,000 objects with Amazon S3, you can on average expect to lose a single object once every 10,000 years.

Part 8: RDS (Relational Database Service)

8.1 What is AWS RDS?

Amazon Relational Database Service (RDS) is a managed service that makes it easy to set up, operate, and scale a relational database in the cloud. It provides cost-efficient and resizable capacity while automating time-consuming administration tasks such as hardware provisioning, database setup, patching, and backups.

8.2 Why Does RDS Exist?

If you try to install MySQL directly onto an EC2 instance, you are responsible for everything:

1. Installing OS patches
2. Backing up the database every day
3. Setting up master-slave replication for high availability
4. Replacing the EC2 instance if the hard drive fails

RDS handles all of this automatically. You just point your Spring Boot application to the RDS endpoint, and AWS takes care of the infrastructure engine.

8.3 RDS Engines Available

RDS supports multiple database engines:

- Amazon Aurora (AWS proprietary MySQL/PostgreSQL, highly performant)
- MySQL
- PostgreSQL
- MariaDB
- Oracle
- Microsoft SQL Server

8.4 Real-World Production Use Cases

- **Transactional Systems:** Storing customer orders, payments, and user profiles.
- **E-Commerce:** Managing product catalogs and shopping cart states.
- **SaaS Platforms:** Managing tenant data and multi-tenant architectures.

8.5 Practical Implementation: Setting up MySQL and Spring Boot

Step 1: Create the RDS Instance

1. Open the RDS Dashboard -> **Create database**.
2. **Creation method:** Standard create.
3. **Engine options:** MySQL.
4. **Templates:** Select **Free tier** or Dev/Test.
5. **DB instance identifier:** `spring-boot-db`.
6. **Master username:** `admin`.
7. **Master password:** Provide a secure password (e.g., `SuperSecret123`).
8. **Public access:** Set to **No** (Security best practice: The database should only be accessible from your private EC2 instances, not the internet).
9. **VPC security group:** Choose "Create new" named `rds-sg`.
10. Click **Create database**. (Takes ~5 minutes).

Step 2: Configure RDS Security Group

We must allow our Spring Boot EC2 instances to talk to the database on port 3306.

1. Open the EC2 Dashboard -> **Security Groups**.
2. Select the `rds-sg` security group.
3. Edit Inbound Rules:
   - **Type:** MySQL/Aurora (3306)
   - **Source:** Type the name of the Security Group assigned to your Spring Boot EC2 instances (e.g., `sg-0abc1234`).
   - Click **Save**.

Step 3: Configure Spring Boot application.properties

Once the database status is "Available", copy the **Endpoint** URL from the RDS console.

In your Spring Boot `src/main/resources/application.properties` file:

```
spring.datasource.url=jdbc:mysql://spring-boot-db.c1abc2xyz.us-east-1.rds.amazonaws.com:3306/mydbname
spring.datasource.username=admin
spring.datasource.password=SuperSecret123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
```

```
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

When you deploy your `.jar` to your EC2 instance, Hibernate will automatically connect to RDS and generate the database tables.

8.6 Advanced RDS Concepts for Production

Multi-AZ Deployments

If your single RDS instance crashes, your application goes down. For production, always enable a **Multi-AZ deployment**. AWS automatically provisions and maintains a synchronous standby replica in a different Availability Zone. The primary DB instance is synchronously replicated across AZs to the standby replica to provide data redundancy. In the event of an infrastructure failure, Amazon RDS performs an automatic failover to the standby.

Read Replicas

If your application runs huge, complex reports read-queries, it will slow down the master database and affect users actively writing orders. Use **Read Replicas** to offload read traffic. AWS creates a read-only copy of your database. You configure your Spring Boot reporting service to connect to the Read Replica URL, taking strain off the master.

8.7 Interview Questions

Q: What is the difference between Multi-AZ and Read Replicas? *Answer:* Multi-AZ is strictly for Disaster Recovery (High Availability). The secondary database cannot be read from; it sits idle waiting for a failover event. Read Replicas are for Performance Scaling (Read-scaling). They are active copies that applications point to run fast SELECT queries.

Q: If you delete an RDS instance, what happens to the backups? *Answer:* Automated backups are deleted when the RDS instance is deleted. However, manual snapshots you've explicitly taken are retained permanently until you manually delete them. AWS prompts you to take a "Final Snapshot" before deleting an instance.

Part 9: Amazon VPC (Networking)

9.1 What is Amazon VPC?

Amazon Virtual Private Cloud (VPC) lets you provision a logically isolated section of the AWS Cloud where you can launch AWS resources in a virtual network that you define. You have complete control over your virtual networking environment, including selection of your own IP address range, creation of subnets, and configuration of route tables and network gateways.

9.2 Why Does It Exist?

Without a VPC, if you launched an EC2 instance or an RDS database, it would sit directly on the open internet, vulnerable to hackers globally. A VPC allows you to build a secure, private fence around your backend servers and databases.

9.3 Core VPC Architecture Concepts

- **VPC (Virtual Private Cloud):** The overarching network boundaries defined by a large CIDR block (e.g., `10.0.0.0/16` gives 65,536 private IPs).
- **Subnets:** Smaller chunks of the VPC deployed inside a specific Availability Zone.
- **Public Subnet:** A subnet that has a route to the internet. This is where you put your Application Load Balancers.
- **Private Subnet:** A subnet that does *not* have direct access to the internet. This is where your Spring Boot EC2 instances and RDS databases go. If they need internet packages (like `yum install java`), they route through a NAT.
- **Internet Gateway (IGW):** The front door attached to the VPC that allows public subnets to communicate with the outside world.
- **NAT Gateway (Network Address Translation):** Placed exactly in the public subnet. It allows resources in *private* subnets to invisibly reach out to the internet to download updates, but prevents the internet from initiating a connection backwards into the private subnet.

9.4 Full Custom VPC Architecture Diagram

```
+-------------------------------------------------------------+
|                Amazon VPC (10.0.0.0/16)                     |
|                                                             |
```

```
|   +-------------------+          [ Internet Gateway (IGW) ] |
|   | Public Subnet     |                 |                   |
|   | 10.0.1.0/24      <------------------+                   |
|   |                   |                                     |
|   | +---------------+ |          +-----------------------+  |
|   | | Load Balancer | |--------->| NAT Gateway (Static IP)| |
|   | +---------------+ |          +-----------------------+  |
|   +---------|---------+                     |               |
|             v                               |               |
|   +-------------------+                     |               |
|   | Private Subnet    |                     |               |
|   | 10.0.2.0/24       |                     |               |
|   |                   |                     v               |
|   |  [ Spring Boot ]----------------[Route out to NAT]       |
|   |  [ EC2 Instance]  |                                     |
|   |         |         |                                     |
|   +---------|---------+                                     |
|             v                                               |
|   +-------------------+                                     |
|   | Private Subnet    |                                     |
|   | 10.0.3.0/24       |                                     |
|   |                   |                                     |
|   | [ RDS MySQL  ]    |                                     |
|   | [ Database   ]    |                                     |
|   +-------------------+                                     |
+-------------------------------------------------------------+
```

9.5 Practical Implementation: Creating a Production VPC

For production, never use the default VPC. Use the **VPC Wizard** to properly secure your stack:

1. Open VPC Dashboard -> **Create VPC**.
2. Select **VPC and more**. This provisions the entire network routing topology instantly.
3. **Name tag auto-generation:** `Spring-Boot-Prod-VPC`.
4. **IPv4 CIDR block:** `10.0.0.0/16`.
5. **Number of Availability Zones (AZs):** 2 (For High Availability).
6. **Number of public subnets:** 2 (For our Load Balancer to span both AZs).
7. **Number of private subnets:** 4 (Two for Spring Boot EC2s, Two for RDS).
8. **NAT gateways:** `In 1 AZ` (Cheaper) or `1 per AZ` (More robust). Required to allow private EC2s to `yum update`.
9. Click **Create VPC**.

From this point forward, when launching EC2 instances or RDS databases, explicitly place them inside the **Private Subnets** of `Spring-Boot-Prod-VPC`.

9.6 Interview Questions

Q: What is the difference between a Security Group and a Network ACL (NACL)? *Answer:* A Security Group acts at the **Instance level**, is Stateful (if traffic is allowed in, the response is automatically allowed out), and rules evaluate all traffic. A NACL acts at the **Subnet level**, is Stateless (you must explicitly allow inbound AND outbound traffic separately), and evaluates rules sequentially starting from rule #100.

Q: Can a private IP from inside a VPC reach the internet? *Answer:* Yes, but only if the subnet routetable sends traffic `0.0.0.0/0` to a NAT Gateway attached to a public subnet holding an Elastic IP.

Part 10: AWS IAM (Identity and Access Management)

10.1 What is IAM?

AWS Identity and Access Management (IAM) helps you securely control access to AWS resources. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources.

10.2 Core IAM Concepts

- **User:** Represents a human or service connecting to AWS. Users have long-term credentials (password or Access Keys).
- **Group:** A collection of users. If you put 10 developers in an "Admin" group, they all instantly receive Administrator privileges.
- **Policies:** JSON documents attached to users/groups/roles defining exactly what actions are allowed/denied.
- **Roles:** Like a "hat" that an AWS service (like EC2) can wear temporarily to gain permissions to speak to another AWS service (like S3). Roles provide short-lived temporary credentials, replacing dangerous hardcoded access keys.

10.3 Understanding IAM JSON Policies

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::user-images-bucket/*"
    }
  ]
}
```

Effect: Is this allowing or denying access? **Action:** What exact API call is being made? (e.g. upload to S3). **Resource:** Which exact bucket/server is allowed?

10.4 Real-World Vulnerability

A massive security mistake in Spring Boot applications: A developer builds an app that uploads files to S3. They generate an Access Key for an IAM User and paste it into `src/main/resources/application.properties`. They accidentally `git push` that file to a public GitHub repository. Within 35 seconds, automated bots scrape GitHub, find the keys, log into AWS, and spin up $50,000 worth of Bitcoin mining EC2 instances.

The Solution: Use IAM Roles attached to EC2.

10.5 Practical Implementation: Allowing EC2 to Access S3 Securely using Roles

Instead of hardcoding AWS credentials in your Spring Boot application to allow it to generate Pre-signed URLs or upload directly via the SDK, AWS provides magical temporary credentials under the hood utilizing IAM Roles.

Step 1: Create the IAM Role

1. Open IAM Dashboard -> **Roles** -> **Create role**.
2. **Trusted entity type:** AWS Service.
3. **Use case:** EC2.
4. Click **Next**.
5. **Add permissions:** Search for `AmazonS3FullAccess` and select it.
6. Click **Next**, Name the role `SpringBoot-EC2-S3-Access-Role`.
7. Click **Create role**.

Step 2: Attach Role to Running EC2 Instance

1. Open the EC2 Dashboard.
2. Check the box next to your running Spring Boot instance.
3. Click **Actions** -> **Security** -> **Modify IAM role**.
4. Select `SpringBoot-EC2-S3-Access-Role` from the dropdown list.
5. Click **Update IAM role**.

Step 3: Application Configuration

Because the EC2 instance now physically "wears" the IAM Role, the underlying AWS SDK operating on the instance intercepts the Role credentials natively from the EC2 instance metadata service. You DO NOT

provide `AWS_ACCESS_KEY_ID` or `AWS_SECRET_ACCESS_KEY` to Spring Boot at all. The SDK authenticates itself instantly and seamlessly.

10.6 Interview Questions

Q: You want to give an external vendor temporary access to your S3 bucket. Should you create an IAM User for them? *Answer:* No. You should use AWS STS (Security Token Service) combined with an IAM Role (or Cross-Account Roles via AssumeRole mechanism) to grant temporary, expiring access rather than creating a permanent user.

Q: If an explicit DENY policy and an explicit ALLOW policy conflict on an IAM User, which one wins? *Answer:* Explicit DENY always trumps ALLOW in AWS IAM policy evaluation logic.

Part 11: AWS Lambda (Serverless Computing)

11.1 What is AWS Lambda and Serverless Architecture?

AWS Lambda is a serverless compute service that runs your code in response to events and automatically manages the underlying compute resources for you.

Serverless Architecture does *not* mean there are no servers. It means that the *cloud provider* (AWS) dynamically manages the allocation and provisioning of servers. You never log into an operating system, you never configure an Auto Scaling Group, and you never apply a Linux security patch. You simply upload a ZIP file containing your code (or write it directly in the browser), and AWS executes it.

11.2 Why Does Lambda Exist?

With EC2 or ECS, if you have a Spring Boot application running, you are paying for that server 24/7--even if no one visits the website at 3 AM.

Lambda exists to provide **true pay-per-use computing**. You are charged exactly for every 1 millisecond your code executes. If your function runs for 200ms once a month, your AWS bill for that compute will be literally $0.00.

Additionally, Lambda solves the problem of idle infrastructure in **Event-Driven Systems**.

11.3 Event-Driven Architecture: How Lambda Works Internally

Lambda is fundamentally event-driven. A Lambda function sits completely dormant (and free) until a specific trigger wakes it up.

1. **The Event:** Something happens in AWS (e.g., a file is uploaded to S3, a message hits an SQS queue, an HTTP request hits API Gateway).
2. **The Trigger:** The AWS service fires off a JSON payload representing that event to Lambda.
3. **Execution Environment:** AWS instantly spins up a micro-container (execution environment), injects your code, and injects the JSON event payload.
4. **Processing & Destruction:** Your code processes the event (e.g., resizes an image). If successful, Lambda spins down the container.
5. **Automatic Scaling:** If 5,000 users upload an image at the exact same second, AWS spins up 5,000 independent, parallel Lambda containers instantly to process them all simultaneously.

```
+-------------------------------------------------------------+
|                  LAMBDA EVENT-DRIVEN FLOW                   |
+-------------------------------------------------------------+
|                                                             |
|   [ Event Source ]      [ AWS Lambda ]        [ Destination]|
|                                                             |
|   Amazon S3      ---->  Lambda scales  ---->  Amazon RDS    |
|   (File upload)         (Reads file,          (Saves result)|
|                         parses data)                        |
|                                                             |
+-------------------------------------------------------------+
```

11.4 When to Use Lambda (Real-World Use Cases)

Lambda is *not* a direct 1-to-1 replacement for a massive monolithic Spring Boot application, though you can run Spring APIs on it. It excels at specific independent tasks:

- **Image/Video Processing:** When a user uploads a `.raw` image to S3, a Lambda triggers instantly to compress it to `.jpg` and create 3 thumbnail sizes.
- **Backend Serverless APIs:** Combining Amazon API Gateway with short-lived Lambda functions (Node.js/Python) to serve React frontend data without provisioning any EC2 servers.
- **Cron Jobs / Scheduled Tasks:** Using Amazon EventBridge to trigger a Lambda function every night at midnight to clean up stale database records in RDS.
- **Data Transformation (ETL):** Reading unstructured data streaming in from Kinesis, transforming it to JSON, and dumping it into another S3 bucket.
- **Notification Routing:** When a specific CloudWatch log error occurs, a Lambda parses the error and fires a message into a Slack Webhook.

11.5 Practical Implementation: Resizing an S3 Image

In a typical React + Spring Boot app, users upload profile pictures to a raw S3 bucket. We want to instantly create a compressed thumbnail using Node.js without putting the CPU strain on our Spring Boot backend.

Step 1: Create the Lambda Function

1. Open AWS Console -> search for **Lambda** -> **Create function**.
2. Select **Author from scratch**.
3. **Function name:** `ImageResizerFunction`.
4. **Runtime:** Select **Node.js 20.x**.

5. **Permissions:** "Create a new role with basic Lambda permissions".
6. Click **Create function**.

Step 2: Write the Lambda Code

In the online code editor for `index.js` (or upload a `.zip` if using external libraries like `sharp` for image processing):

```
const AWS = require('aws-sdk');
const s3 = new AWS.S3();

exports.handler = async (event) => {
    // 1. The event trigger passes a massive JSON object detailing the S3
upload
    console.log("Event received: ", JSON.stringify(event));

    // 2. Extract the bucket name and file name (key)
    const bucket = event.Records[0].s3.bucket.name;
    const key =
decodeURIComponent(event.Records[0].s3.object.key.replace(/\+/g, ' '));

    console.log(`Processing file ${key} from bucket ${bucket}`);

    try {
        // 3. Download the raw file from S3
        const params = { Bucket: bucket, Key: key };
        const rawImage = await s3.getObject(params).promise();

        // 4. (Pseudo-code) Perform image resizing using a library like 'sharp'
        // const resizedBuffer = await sharp(rawImage.Body).resize(200,
200).toBuffer();

        console.log(`Successfully processed ${key}. Ready to save thumbnail.`);

        return { statusCode: 200, body: 'Image processed successfully' };
    } catch (err) {
        console.error("Error processing file", err);
        throw err;
    }
};
```

Step 3: Add S3 Permissions to the Lambda Role

By default, the Lambda execution role can only write logs to CloudWatch. It cannot read from S3.

1. In the Lambda function -> **Configuration** tab -> **Permissions**.
2. Click the Execution Role name to open the IAM Dashboard.
3. Click **Add permissions** -> **Attach policies**.
4. Search for `AmazonS3ReadOnlyAccess` (or full access if uploading the thumbnail) and attach it.

Step 4: Configure the Event Trigger

1. Go back to your Lambda function console.
2. In the "Function overview" diagram, click **+ Add trigger**.
3. **Trigger configuration:** Select **S3**.
4. **Bucket:** Select your raw images bucket (e.g., `user-profile-images-2026`).
5. **Event type:** `All object create events` (triggers when a `PUT` occurs).
6. **Suffix:** `.jpg` (Optional, to only trigger on JPEG files).
7. Click **Add**.

Now, whenever your React app uploads an image to `user-profile-images-2026`, AWS will instantly fire a payload to `ImageResizerFunction`.

11.6 Common Mistakes Developers Make

- **Cold Starts:** If a Lambda function hasn't been accessed in a while, AWS destroys the micro-container. The next time it's triggered, AWS has to allocate a container and load the runtime (e.g., the JVM). This can add seconds of latency (a "Cold Start"). This is a huge issue for Spring Boot / Java on Lambda. Node.js and Python are typically preferred for user-facing APIs due to faster startup times.
- **Infinite Loops:** If a Lambda function processes a file from Bucket A, resizes it, and saves it *back to Bucket A*, it will trigger the Lambda again, creating an infinite loop that can cost thousands of dollars within minutes. *Always save processed files to a different bucket!*
- **Long-Running Processes:** Lambda has a hard timeout limit of 15 minutes. It is strictly not for hours-long batch processing.

11.7 Interview Questions

Q: What is the difference between EC2 and Lambda? *Answer:* EC2 is Infrastructure as a Service (IaaS); you provision virtual OS instances, manage scaling rules, and pay by the hour. Lambda is Function as a Service (FaaS); you provision only code, scaling is entirely invisible, and you pay by the millisecond of execution.

Q: What is a Lambda "Cold Start" and how do you mitigate it for Java applications? *Answer:* A cold start occurs when a Lambda container needs to be spun up from scratch. In Java, starting the JVM and

loading dependency injection frameworks takes a long time. You can mitigate this using GraalVM to compile Java ahead-of-time (AOT) into a native binary, or by using AWS "Provisioned Concurrency" to keep a set amount of containers perpetually warm.

Part 12: Docker Deployment (ECS & ECR)

12.1 What are Docker and ECS?

Docker allows you to package your Spring Boot or React application, along with its runtime, system tools, and system libraries, into a standardized unit for software development called a *Container*.

Amazon Elastic Container Service (ECS) is a highly scalable, high-performance container orchestration service that supports Docker containers. It allows you to easily run and scale containerized applications on AWS without having to manage the underlying EC2 instances directly (using AWS Fargate serverless compute).

Amazon Elastic Container Registry (ECR) is a fully managed container registry that makes it easy to store, manage, and deploy Docker container images (AWS's private version of Docker Hub).

12.2 Why Use Containers on AWS?

Without Docker, your EC2 Java deployments are subject to the "it works on my machine" problem. Perhaps your local machine has Java 17 installed, but the EC2 instance has Java 11. With Docker, the exact same environment runs locally as it does on AWS. ECS then takes those immutable containers and scales them up and down seamlessly, handling restarts if a container crashes.

12.3 Practical Implementation: Deploying Spring Boot to ECS

Step 1: Create a Dockerfile

In the root of your Spring Boot project, create a file named `Dockerfile`:

```
# Use official Java 17 runtime as a parent image
FROM eclipse-temurin:17-jre-alpine
# Set the working directory
WORKDIR /app
# Copy the executable jar
COPY target/my-spring-app-0.0.1-SNAPSHOT.jar app.jar
# Expose the API port
EXPOSE 8080
# Run the application
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Step 2: Push Image to AWS ECR

1. Open AWS Console -> search for **Elastic Container Registry (ECR)** -> **Create repository**. Name it `spring-boot-repo`.
2. Open your local terminal, authenticate the Docker client to your ECR registry, build, tag, and push the image. AWS provides these exact commands when you click **View push commands** in the console:

```
# Authenticate
aws ecr get-login-password --region us-east-1 | docker login --username AWS --
password-stdin 123456789012.dkr.ecr.us-east-1.amazonaws.com

# Build the local Docker image
docker build -t spring-boot-repo .

# Tag it for AWS
docker tag spring-boot-repo:latest 123456789012.dkr.ecr.us-east-
1.amazonaws.com/spring-boot-repo:latest

# Push to ECR
docker push 123456789012.dkr.ecr.us-east-1.amazonaws.com/spring-boot-
repo:latest
```

Step 3: Deploy to ECS (Using Fargate)

1. Open **Elastic Container Service (ECS)** -> **Task Definitions** -> **Create new Task Definition**.
2. Select **AWS Fargate** (Serverless compute for containers, no EC2 management!).
3. **Container details:** Name `spring-boot-container`, Image URI: (copy from ECR), Port mappings: `8080`.
4. Click **Create**.
5. Go to **Clusters** -> **Create Cluster**. Name it `spring-boot-cluster`.
6. Inside your cluster, create a **Service**.
7. **Launch type:** Fargate.
8. **Task definition:** Select the one you just created.
9. **Networking:** Select your VPC and Subnets. Assign a Security Group that allows inbound port 8080. Enable **Public IP**.
10. Click **Create Service**. ECS will now pull your Docker image and run it globally.

12.4 Common Mistakes

- **OOM (Out of Memory) Kills:** Allocating 512MB of RAM to a Fargate task running a heavy Spring Boot application will crash instantly. Ensure your Fargate task has at least 1GB - 2GB of RAM defined.

Part 13: CI/CD Pipeline (CodePipeline)

13.1 What is CI/CD?

Continuous Integration and Continuous Deployment (CI/CD) automates your software delivery process. Instead of manually building a `.jar` locally and copying it to a server, every time you `git push` code to GitHub, AWS automatically downloads the code, runs the tests, compiles it, builds a Docker image, and deploys it to your ECS cluster or EC2 instances.

13.2 CI/CD Stack on AWS

- **AWS CodeCommit:** Fully-managed source control service (like GitHub).
- **AWS CodeBuild:** Fully managed continuous integration service that compiles source code, runs tests, and produces software packages.
- **AWS CodeDeploy:** Automates code deployments to any instance, including Amazon EC2 instances and on-premises servers.
- **AWS CodePipeline:** Fully managed continuous delivery service that automates your release pipelines for fast and reliable updates. It glues everything above together.

13.3 Practical Implementation: GitHub to EC2 Pipeline

Step 1: Prepare the Add `appspec.yml`

CodeDeploy requires an `appspec.yml` file in your project root to know how to install the app.

```yaml
version: 0.0
os: linux
files:
  - source: /target/app.jar
    destination: /home/ec2-user/
hooks:
  ApplicationStop:
    - location: scripts/stop_server.sh
      timeout: 300
      runas: root
  ApplicationStart:
    - location: scripts/start_server.sh
```

```
    timeout: 300
    runas: root
```

You must also include `start_server.sh` and `stop_server.sh` shell scripts in your repo.

Step 2: Create a CodePipeline

1. Go to AWS Console -> **CodePipeline** -> **Create pipeline**.
2. **Name:** `spring-boot-pipeline`.
3. **Source provider:** Select **GitHub (Version 2)**. Connect to your GitHub account and select your repository and `main` branch.
4. **Build provider:** Select **AWS CodeBuild**.
   - Create a new Build Project.
   - Select Managed image (Ubuntu / standard).
   - In your repo, you must include a `buildspec.yml` telling CodeBuild how to compile Java:

     ```
     version: 0.2
     phases:
       build:
         commands:
           - mvn clean package
     artifacts:
       files:
         - target/app.jar
         - appspec.yml
         - scripts/**/*
     ```

5. **Deploy provider:** Select **AWS CodeDeploy**.
   - You must quickly create a CodeDeploy application and Deployment Group pointing to your EC2 instances (identified by tags, e.g., `Env:Prod`).
6. Click **Create pipeline**.

Now, every single `git push` to `main` will automatically trigger a build and zero-downtime deployment!

13.4 Interview Questions

Q: In Blue/Green deployments using CodeDeploy, what does Blue and Green represent? *Answer:* Blue represents the currently running production environment. Green represents the newly deployed

version. Traffic is gradually shifted from Blue to Green. If the Green environment fails health checks, traffic is instantly routed back to Blue.

Part 14: AWS CloudWatch (Monitoring)

14.1 What is CloudWatch?

Amazon CloudWatch is a monitoring and observability service. It provides you with data and actionable insights to monitor your applications, respond to system-wide performance changes, and optimize resource utilization.

14.2 Core Concepts

- **Metrics:** Data points related to your resources (e.g., EC2 CPU utilization, ALB HTTP 500 error count, RDS disk space remaining).
- **Alarms:** Watch a single metric over a time period and perform one or more actions (like sending an email or triggering Auto Scaling) if the metric exceeds a threshold.
- **Logs:** Collect, monitor, and store log files from your EC2 instances, AWS CloudTrail, Route 53, and other sources.

14.3 Real-World Production Use Cases

- Creating dashboards for the CEO displaying the exact number of active users connected to the Load Balancer.
- Setting an Alarm to page the DevOps team at 3 AM if the Spring Boot API returns more than ten 500 Internal Server Errors in a 5-minute window.
- Viewing raw Spring Boot application logs centrally, rather than SSHing into 10 different EC2 instances separately to read `app.log`.

14.4 Practical Implementation: Viewing Logs and Creating an Alarm

Step 1: View Logs (Fargate/ECS Auto-Integration)

If you deploy your Spring Boot container using ECS Fargate, logging is handled instantly.

1. Open the ECS cluster and click your Task.
2. Under the **Logs** tab, every `System.out.println()` and `logger.info()` from your Java app streams directly to CloudWatch Logs automatically. No configuration needed!

Step 2: Create an Alarm for CPU Spike

1. Go to **CloudWatch** -> **Alarms** -> **Create alarm**.
2. Click **Select metric**.
3. Go to **EC2** -> **Per-Instance Metrics** -> Find your production instance and select `CPUUtilization`.
4. **Conditions:** Select **Greater/Equal**, Threshold value: **80%**. Period: **5 minutes**.
5. **Actions:** Under Notification, select "Create a new topic", enter your email address (e.g., `admin@company.com`).
6. Name the alarm `High-CPU-Alert` and click **Create alarm**. You will receive an email confirmation. Now, whenever the server spikes, you will be notified instantly.

14.5 Common Mistakes

- **Logging Sensitive Data:** Logging credit card numbers or raw unhashed passwords to standard out in Spring Boot. CloudWatch logs are stored in plain text and accessible by anyone in the company with CloudWatch Read access.
- **Unbounded Log Retention:** By default, CloudWatch Logs are retained "Never expire". For a high-traffic API, this will cost thousands of dollars a month in storage fees after a year. Always configure existing log groups to have a retention period of 14 or 30 days.

Part 15: Full Production Architecture

This architecture represents the gold standard for deploying a highly available, scalable, fault-tolerant React + Spring Boot + MySQL full-stack application.

15.1 Enterprise Cloud Architecture Diagram

```
+-----------------------------------------------------------------------------
-----------+
|                                   INTERNET Users
|
+-----------------------------------------------------------------------------
-----------+
           |                                        |
           v  (Static React App)                    v  (API Requests)
    +-----------------------+                +-----------------------+
    |    AWS CloudFront     |                |  Amazon API Gateway   |
    | (Global Edge Network) |                | (Rate Limiting/Auth)  |
    +-----------------------+                +-----------------------+
           |      ^                                 |
           |      |                                 v
           v      |                        [Application Load Balancer]
       +-----------------+                          |
       |    Amazon S3    |                          | (Routes to healthy
instances)
       | (Hosts React &  |                          |
       |  Static Assets) |                          v
       |                 |                  +-------------------+
       |  [File Uploads]-|----------------->| Auto Scaling Group|
       +-----------------+                  |  [Spring Boot]    | <----+
               |                            |  [Spring Boot]    |      | Logs
               v (Event Trigger)            +-------------------+      |
       +-----------------+             (Private Connectivity) |        |
       |   AWS Lambda    |                                    |        |
       | (Image Resizer, |        +-----------------------+  |        |
       |  Webhook pushes)|        v                       v  v        v
       +-----------------+  +-------------------+  +-------------------+
                            |    Amazon RDS     |  | Amazon CloudWatch |
                            |  (Master MySQL)   |  | (Logs & Alarms)   |
                            +-------------------+  +-------------------+
                                      | (Sync Replication)
                                      v
                            +-------------------+
```

```
                                  | Amazon RDS (Standby)|
                                  | (Different AZ)    |
                                  +-------------------+
```

15.2 Component Breakdown

1. **Amazon S3:** Stores the compiled, optimized build of your React frontend app. Also stores raw user file uploads.
2. **AWS CloudFront:** A globally distributed CDN that caches the React S3 bucket contents across hundreds of Edge Locations globally, delivering sub-millisecond load times.
3. **AWS Lambda:** Short-lived Serverless functions that trigger instantly when an image is uploaded to S3 to create thumbnails without placing CPU burden on the backend.
4. **Amazon API Gateway:** Exposes your backend REST APIs. Protects internal microservices by enabling API Keys, Throttling (Rate Limiting), and integrating with AWS Cognito for authentication.
5. **Application Load Balancer:** Distributes API requests received from the API Gateway evenly across multiple EC2 or ECS Spring Boot instances.
6. **Auto Scaling Group / ECS Fargate:** The compute layer running your Spring Boot application across *multiple* Availability Zones. If CPU > 70%, it scales out automatically.
7. **Amazon RDS MySQL (Multi-AZ):** Stores all user and application data. If the master database fails, traffic instantly shifts to the synchronous standby database in a different AZ with zero data loss.
8. **CloudWatch:** Aggregates logs from Spring Boot and Lambda, monitors Error Rates, and tracks metric alarms (e.g., if RDS CPU hits 90%).

Part 16: Interview Preparation (50+ AWS Questions)

The following real-world interview questions prepare developers for Senior Cloud Engineering positions.

Core Concepts

1. What is AWS? Amazon Web Services (AWS) is a highly scalable, flexible, and cost-effective cloud computing platform that provides reliable and scalable cloud computing services globally. **2. What are the key elements of AWS global infrastructure?** Regions, Availability Zones (AZs), and Edge Locations. **3. What is an Availability Zone?** An isolated location within an AWS Region consisting of one or more discrete data centers. **4. What is a Region?** A geographical area containing two or more Availability Zones. **5. What is an Edge Location?** Endpoints used by AWS CloudFront caching systems to deliver content with low latency globally. **6. What are the different types of cloud services?** IaaS (Infrastructure), PaaS (Platform), SaaS (Software). **7. Explain the AWS Shared Responsibility Model.** AWS is responsible for security *of* the cloud (physical data centers, network). The customer is responsible for security *in* the cloud (IAM users, configuring Security Groups, encrypting data).

Compute & Scalability

8. What is Amazon EC2? Elastic Compute Cloud provides scalable virtual servers in the cloud. **9. How do you secure an EC2 instance?** Using Security Groups (firewall), Key Pairs (SSH), and placing it in a Private Subnet. **10. What is an AMI?** Amazon Machine Image. A template that provides the OS and applications required to launch an EC2 instance. **11. What is an Elastic IP address?** A static, public IPv4 address that does not change when an EC2 instance stops and starts. **12. On-Demand vs Reserved vs Spot instances?** On-Demand: Pay by the hour, no commitment. Reserved: 1-3 year commitment for massive discount. Spot: Bid on unused AWS capacity for up to 90% discount, but can be terminated with a 2-minute warning. **13. What is AWS Auto Scaling?** Automatically adjusts the number of EC2 instances based on traffic or load (e.g., CPU utilization). **14. Horizontal vs Vertical Scaling in AWS?** Horizontal (Scale Out/In) adds/removes EC2 instances. Vertical (Scale Up/Down) increases/decreases instance size (e.g., `t3.micro` to `c5.xlarge`). AWS Auto Scaling handles Horizontal scaling. **15. What is a Load Balancer (ELB)?** Distributes incoming application traffic across multiple EC2 instances to increase fault tolerance. **16. ALB vs NLB?** Application Load Balancer operates at Layer 7 (HTTP/HTTPS) and routes based on URL paths. Network Load Balancer operates at Layer 4 (TCP/UDP) for extreme performance and static IPs.

Storage & Database

17. What is Amazon S3? Simple Storage Service. A highly scalable, durable object storage service. **18. What is the durability of S3?** 99.999999999% (11 nines). **19. What is a Bucket Policy?** A JSON

document attached to an S3 bucket that controls access permissions across the entire bucket. **20. Difference between S3 and EBS?** S3 is object storage (like a limitless hard drive in the sky access via API). EBS (Elastic Block Store) is block storage representing a literal hard drive directly attached to a specific EC2 instance. **21. What is an S3 Pre-signed URL?** A temporary URL generated using AWS credentials that grants temporary access to download or upload an object securely. **22. What is Amazon RDS?** Relational Database Service. A managed service for MySQL, PostgreSQL, Oracle, etc. **23. What is Amazon DynamoDB?** A fully managed, fast NoSQL database service with single-digit millisecond performance at any scale. **24. Multi-AZ vs Read Replicas in RDS?** Multi-AZ is for synchronous Disaster Recovery (High Availability). Read Replicas are for asynchronous Read Scaling (Performance).

Networking (VPC)

25. What is Amazon VPC? Virtual Private Cloud. A private, isolated section of the AWS cloud where you define your networking architecture. **26. Public vs Private Subnet?** A public subnet has a route to an Internet Gateway (IGW). A private subnet does not. **27. What is an Internet Gateway (IGW)?** Allows compute instances in a public subnet to connect to the internet. **28. What is a NAT Gateway?** Allows instances in a private subnet to securely reach the internet to download updates without allowing inbound traffic from the internet. **29. Security Group vs Network ACL (NACL)?** Security groups operate at the Instance level (Stateful). NACLs operate at the Subnet level (Stateless).

Security & IAM

30. What is IAM? Identity and Access Management. Controls secure access to AWS services. **31. Difference between IAM User and IAM Role?** A User has permanent credentials (password, access key). A Role provides temporary, short-lived credentials intended to be assumed by an EC2 instance or service. **32. What is an IAM Policy?** A JSON document defining permissions (Allow/Deny actions on resources). **33. What is AWS CloudTrail?** A service that logs every single API call made in your AWS account for auditing and compliance. **34. How do you securely pass AWS credentials to a Spring Boot app on EC2?** Attach an IAM Role directly to the EC2 instance. The AWS SDK in Spring Boot automatically assumes the role via Instance Metadata. *Never* hardcode access keys.

Docker, ECS, and Serverless

35. What is Amazon ECS? Elastic Container Service. A managed container orchestration service to run Docker containers. **36. ECS EC2 Launch Type vs ECS Fargate?** With EC2 Launch Type, you manage the underlying EC2 instances executing the containers. With Fargate, AWS completely abstracts the servers; you just specify CPU/RAM for the container (Serverless Compute). **37. What is Amazon ECR?** Elastic Container Registry. AWS's private managed Docker container registry. **38. What is AWS Lambda?** A Serverless compute service that runs code (Java, Node.js, Python) in response to events and automatically manages the computing resources required.

CI/CD & Monitoring

39. What are the Code suite services? CodeCommit (Git source control), CodeBuild (compiling/testing), CodeDeploy (deploying to EC2), CodePipeline (orchestrates the flow). **40. What is Amazon CloudWatch?** A monitoring service for logging, metrics, and alarms. **41. How does Spring Boot log to CloudWatch?** Standard output (`System.out.println` or Logback console appender) running inside an ECS Fargate container automatically streams directly to a CloudWatch Log Group.

Serverless (Lambda)

42. What is AWS Lambda? A Serverless compute service that runs code (Java, Node.js, Python) in response to events and automatically manages the computing resources required. **43. What does "Serverless" architecture mean?** It means the cloud provider dynamically manages the allocation and provisioning of servers. You only pay for exactly what you use right down to the millisecond of execution time, rather than paying for idle EC2 servers 24/7. **44. What is a Lambda trigger?** An AWS service or event (like an S3 file upload, API Gateway HTTP request, or EventBridge cron schedule) that invokes a Lambda function to execute. **45. Why might you use Node.js or Python instead of Java for a Lambda function?** Lambda functions suffer from "Cold Starts"--the time it takes for AWS to provision a new container. A JVM takes significantly longer to start than Node.js or Python, leading to high latency spikes on the first invocation.

Scenario-Based Technical Questions

46. A Spring Boot instance keeps crashing with OutOfMemoryExceptions. How do you diagnose it? Check CloudWatch Logs for the exact stack trace. Check CloudWatch Metrics to map memory spikes. Increase EC2/ECS memory limits if under-provisioned. **47. Your Application Load Balancer returns a 502 Bad Gateway. Why?** The backend EC2 instances are offline, misconfigured, or not returning a valid HTTP payload within the ALB timeout period. **48. Users are complaining the React app loads slowly in Asia, but fast in the US. Solution?** Host the React app statically in an S3 bucket and distribute it globally via the CloudFront CDN edge network. **49. Your EC2 instance cannot connect to the internet to download Java. Why?** If in a private subnet, there is no NAT Gateway configured in the route table. If in a public subnet, missing an Elastic IP or the IGW is missing from the route table. **50. How would you handle processing 50,000 images uploaded to S3 by users simultaneously?** Use AWS Lambda triggered by S3 PutObject events. Lambda will automatically horizontally scale out to 50,000 parallel Execution Environments instantly, processing them all concurrently without burdening the Spring Boot backend servers.

Part 17: AWS Production Best Practices

To confidently run real-world enterprise applications in the cloud, implement these five pillars of AWS Architecture:

1. Security

- Use IAM Roles instead of Access Keys everywhere.
- Enable MFA on root and administrator accounts.
- Encrypt your RDS databases and S3 buckets at rest using AWS KMS.
- Keep EC2 instances entirely in Private Subnets with no public IPs.

2. Reliability (High Availability)

- Always launch backend EC2/ECS instances into an Auto Scaling Group across at least 2 Availability Zones.
- Always run production RDS in Multi-AZ mode to survive database failure.
- Offload static assets (images, frontend React apps) to S3 + CloudFront so they never impact backend reliability.

3. Performance Efficiency

- Implement read replicas on RDS for heavy reporting queries.
- Optimize Java JVM memory settings (Heap size) specifically for your instance type.
- Use CloudFront to globally cache API payloads closer to the end user.

4. Cost Optimization

- Set up AWS Billing Alarms to prevent bill shock.
- For non-production environments (Dev/Test), use Spot Instances to save 90% on compute cost.
- Configure S3 Lifecycle rules to move 30-day-old files to cheaper S3 Glacier storage.

5. Operational Excellence

- Adopt Infrastructure as Code (Terraform or AWS CloudFormation) so your entire architecture is reproducible via code instead of clicking through the AWS Console manually.
- Consolidate all central application errors into CloudWatch Alarms that ping Slack or PagerDuty instantly.